DIPLOMADO EN INGENIERÍA DE SOFTWARE ASISTIDA POR INTELIGENCIA ARTIFICIAL

ROBERTH MAURICIO LÓPEZ ZORRILLA

Módulo 2. Arquitectura y Diseño de Aplicaciones con IA

UNIVERSIDAD SANTIAGO DE CALI

FACULTAD DE INGENIERÍA

INGENIERÍA DE SISTEMAS

CALI

2026

CONTENIDO

FIGURAS 3

TABLAS 3

INTRODUCCIÓN 4

1. DESCRIPCIÓN DEL PROYECTO 5

2. OBJETIVOS DE LA PROPUESTA ARQUITECTÓNICA 7

3. DIAGRAMA DE CONTEXTO 9

4. DIAGRAMA DE COMPONENTES Y ARQUITECTURA PROPUESTA 11

5. MODELO DE DATOS 14

6. ESPECIFICACIÓN INICIAL DE LA API 18

7. DECISIONES ARQUITECTÓNICAS 22

8. DISEÑO DE LA INTEGRACIÓN DE IA 23

9. SUSTITUCIÓN DEL PROVEEDOR DE IA Y CONTINUIDAD OPERATIVA 29

10. SEGURIDAD, PRIVACIDAD Y CONTROL DE ACCESO 34

11. VALIDACIÓN PREVISTA DEL DISEÑO 36

12. CONCLUSIONES 37

REFERENCIAS BIBILIOGRÁFICAS 38

FIGURAS

FIGURA 1 DIAGRAMA DE CONTEXTO (C4, NIVEL 1) 9

FIGURA 2 ARQUITECTURA EN CAPAS CON PUERTOS Y ADAPTADORES (HEXAGONAL LIGERA) 11

FIGURA 3 MODELO DE DATOS 14

FIGURA 4 FLUJO DE IMPORTACIÓN ASISTIDA DE PRODUCTOS MEDIANTE IA 24

TABLAS

TABLA 1 ACTORES, SISTEMA Y RESPONSABILIDAD 10

TABLA 2 COMPONENTES PROPUESTOS 12

TABLA 3 ENTIDADES PRINCIPALES 15

TABLA 4 ENDPOINTS 18

TABLA 5 DECISIONES ARQUITECTÓNICAS 22

TABLA 6 PROVEEDORES COSTO-BENEFICIO 30

TABLA 7 ELEMENTOS QUE PODRÍAN CAMBIAR 32

TABLA 8 VALIDACIÓN PREVISTA DEL DISEÑO 36

INTRODUCCIÓN

Los establecimientos dedicados a la reparación de celulares, computadores y consolas suelen manejar información relacionada con sus productos, repuestos, movimientos de inventario y servicios técnicos. Cuando estos registros se distribuyen entre cuadernos, archivos independientes y conversaciones por medio de chat de redes sociales, consultar la información puede requerir tiempo adicional y dificultar el seguimiento de las actividades.

Se plantea este proyecto como una aplicación web orientada a organizar parte de estos procesos en un mismo sistema. La propuesta contempla administración de productos, el registro de movimientos básicos de inventario y la gestión de órdenes de reparación mediante un código que permita consultar su estado. El diseño también considera una función de inteligencia artificial (IA) para apoyar la preparación de datos de productos provenientes de archivos tabulares. Esta función se concibe como una herramienta de asistencia, podrá proponer una organización de los campos y sugerir descripciones a partir de la información suministrada, pero no deberá inventar precios, cantidades o compatibilidades. Antes de guardar los datos, una persona tendrá que revisar y aprobar la propuesta.

El documento presenta una arquitectura inicial, los componentes principales, un modelo de datos, una especificación preliminar de la API y las decisiones consideradas para incorporar IA. Se utilizan como apoyo documentación técnica oficial, estándares y publicaciones relacionadas con arquitectura de software, persistencia, diseño de interfaces de programación y gestión de riesgos de IA.

1. DESCRIPCIÓN DEL PROYECTO

1.1 Situación identificada

A partir de la observación del proceso de negocio y del conocimiento previo de las actividades relacionadas con los servicios técnicos, se identificaron situaciones que pueden dificultar la organización de la información:

- El registro y la actualización de productos y repuestos pueden requerir trabajo manual.
- Los clientes consultan repetidamente por mensajería el estado de las reparaciones.
- La información sobre productos, precios y disponibilidad puede encontrarse distribuida en diferentes registros.
- Los negocios suelen utilizar publicaciones en estados de mensajería para promocionar servicios y productos, lo que puede dificultar la consulta posterior de información publicada.
- Los registros manuales pueden hacer más lenta la búsqueda de antecedentes de un producto o servicio.

Estas situaciones constituyen el punto de partida del proyecto y fueron identificadas a partir de entrevistas relacionadas con el funcionamiento del negocio, complementadas con el conocimiento propio de sus procesos operativos. La información recopilada permitió reconocer las principales necesidades asociadas con la gestión del inventario, el seguimiento de las reparaciones y la atención de consultas de los clientes. A partir de este diagnóstico, se establecieron las necesidades iniciales que orientan la definición de los requisitos y las funcionalidades de GestorTech.

1.2 Propuesta de solución

GestorTech propone centralizar determinadas actividades en una aplicación web con dos áreas principales:

Área administrativa: destinada a las personas autorizadas del establecimiento para gestionar productos, registrar movimientos básicos de inventario y administrar las órdenes de reparación.

Área de consulta para clientes: destinada a consultar el estado de una reparación mediante un código de seguimiento. La consulta mostrará únicamente la información definida como pública para ese propósito.

La propuesta incluye, además, un asistente de IA para apoyar la preparación de datos de productos antes de incorporarlos al catálogo.

1.3 Alcance funcional

La primera versión contempla las siguientes funcionalidades:

- Registro, consulta y actualización de productos.
- Registro de entradas y salidas de inventario.
- Creación de órdenes de reparación con un código único de seguimiento.
- Actualización del estado de una reparación y registro de su historial.

- Consulta del avance de una reparación mediante el código correspondiente.
- Preparación asistida de datos de productos mediante IA, con revisión humana obligatoria.

1.4 Elementos excluidos del alcance inicial

Para mantener el proyecto dentro de un alcance manejable, se excluyen de esta propuesta inicial:

- Contabilidad completa, facturación electrónica y gestión tributaria.
- Procesamiento de pagos y administración de transacciones financieras.
- Automatización integral de WhatsApp.
- Una plataforma especializada en la venta de videojuegos.
- El guardado automático de datos generados por IA sin revisión humana.
- La integración obligatoria con un proveedor específico de inteligencia artificial.

Estas exclusiones permiten concentrar el diseño en el inventario, las reparaciones y la preparación asistida de datos.

2. OBJETIVOS DE LA PROPUESTA ARQUITECTÓNICA

2.1 Objetivo general

Definir una arquitectura inicial para GestorTech que organice la gestión de inventario y el seguimiento de reparaciones técnicas, incorporando una función opcional de inteligencia artificial para apoyar la preparación de datos de productos bajo supervisión humana.

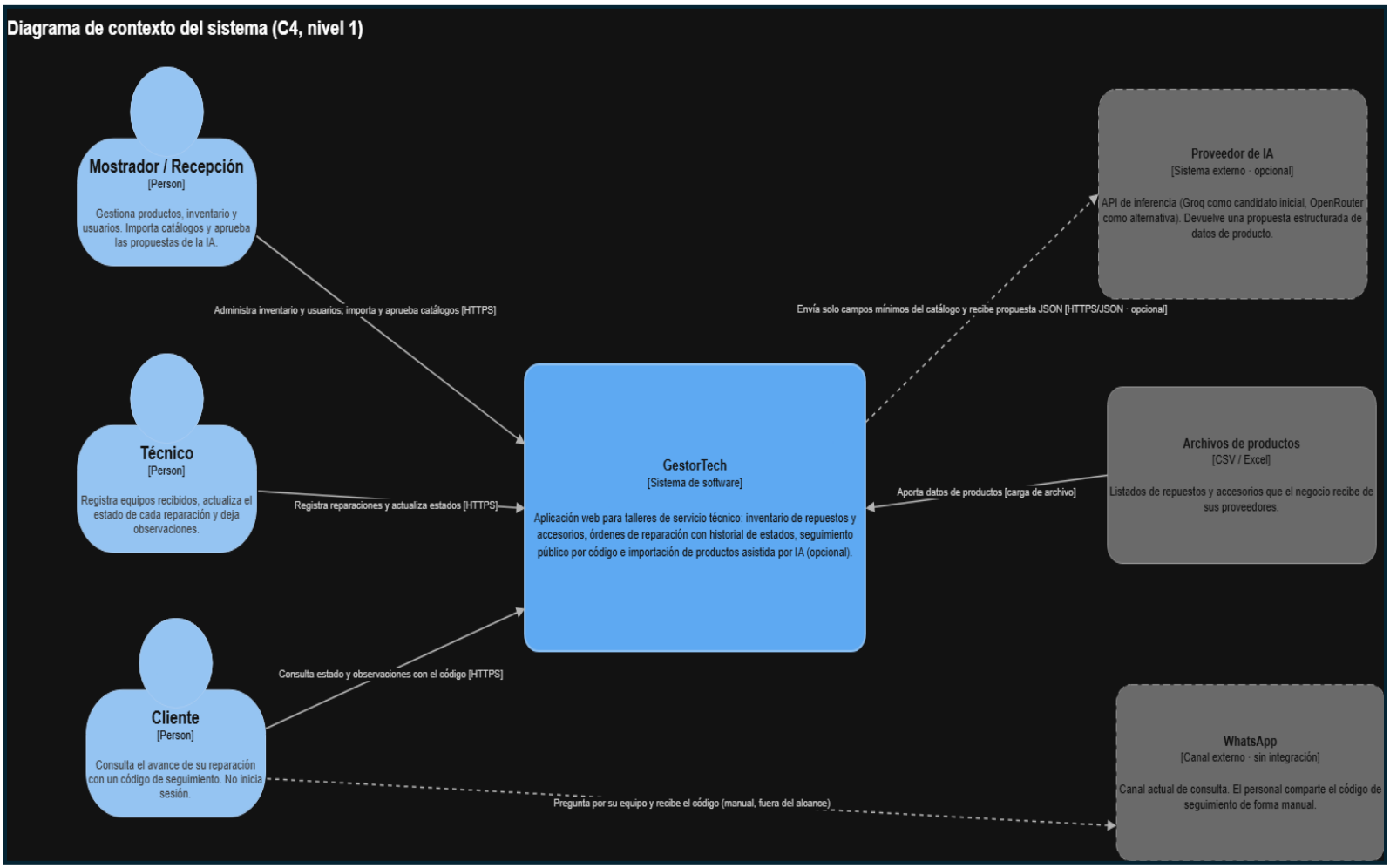
2.2 Objetivos específicos

- Identificar los actores, componentes y responsabilidades que intervienen en la solución propuesta.
- Definir un modelo de datos inicial que represente los productos, los movimientos de inventario, las reparaciones y su historial.
- Especificar las operaciones principales de una API para facilitar la comunicación entre la interfaz web y la lógica del sistema.
- Establecer criterios para integrar un proveedor de IA mediante una interfaz independiente, sin vincular el resto de la aplicación a una implementación particular.
- Incorporar consideraciones de validación, control de acceso, privacidad y manejo de errores en el diseño inicial.

Estos objetivos describen los resultados de diseño que se pretenden alcanzar, no constituyen evidencia de que todas las funcionalidades ya se encuentren desarrolladas.

3. DIAGRAMA DE CONTEXTO

Figura 1 Diagrama de contexto (C4, nivel 1)



El diagrama de contexto representa a GestorTech como el sistema central y muestra las relaciones con el administrador, el técnico y el cliente. También identifica tres elementos externos: los archivos CSV o Excel que aportan los datos del catálogo, el proveedor de IA opcional y WhatsApp como canal que actualmente se utiliza sin integración automática. El modelo C4 de

contexto sirve para delimitar el sistema y mostrar sus usuarios y dependencias externas sin entrar todavía en los detalles internos de implementación (Brown, s. f.).

En el diagrama, el administrador gestiona el inventario y los usuarios, además de importar y aprobar catálogos. El técnico registra las reparaciones y actualiza sus estados. El cliente consulta el avance mediante un código de seguimiento. Los archivos de productos se cargan en la aplicación; el proveedor de IA recibe únicamente los campos necesarios y devuelve una propuesta estructurada. WhatsApp aparece como canal externo sin integración: el personal comparte manualmente el código de seguimiento.

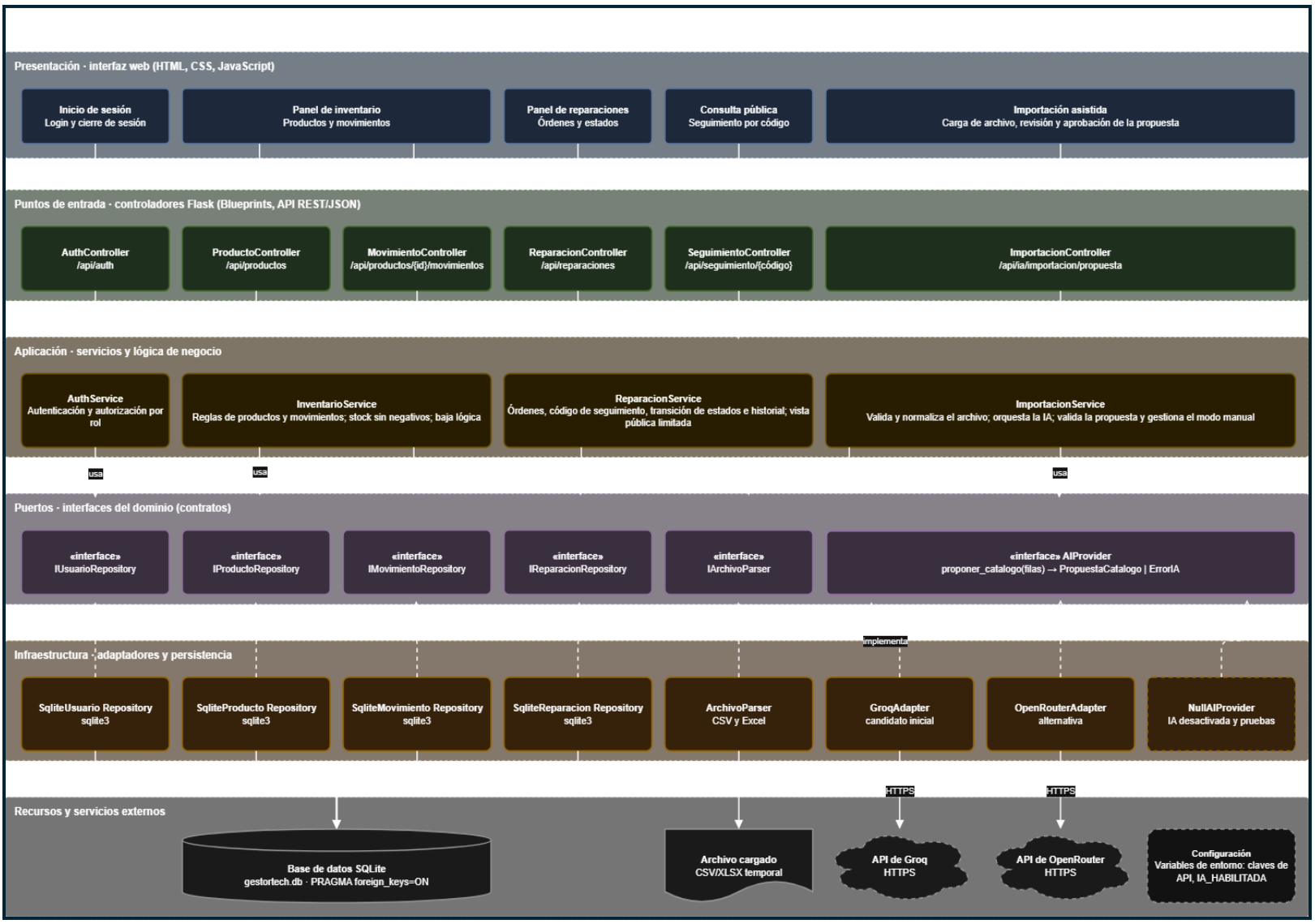
Tabla 1 Actores, sistema y responsabilidad

Actor o sistema	Responsabilidad dentro de la propuesta
Administrador del establecimiento	Gestionar productos, consultar inventario y administrar las reparaciones según sus permisos.
Técnico	Consultar las órdenes asignadas y actualizar el avance de las reparaciones de acuerdo con sus permisos.
Cliente	Consultar el estado de una reparación utilizando el código de seguimiento.
Proveedor externo de IA	Procesar solicitudes de asistencia para organizar o proponer información de productos, actualizar inventario y consultar existencias, si se habilita esta integración.

La aplicación centraliza la lógica del negocio y controla qué información puede consultar o modificar cada actor. La separación entre GestorTech y el proveedor externo busca limitar el alcance de la integración. Si la IA no está disponible, las operaciones principales de inventario y reparación deberán seguir funcionando mediante los componentes propios de la aplicación.

4. DIAGRAMA DE COMPONENTES Y ARQUITECTURA PROPUESTA

Figura 2 Arquitectura en capas con puertos y adaptadores (hexagonal ligera)



4.1 Enfoque arquitectónico

Se propone una aplicación web desarrollada con Python y Flask, con una interfaz construida mediante HTML, CSS y JavaScript, y una base de datos SQLite para la primera versión.

Flask proporciona las herramientas para construir aplicaciones web y permite organizar las rutas y funcionalidades mediante mecanismos como los Blueprints (Pallets Projects, s. f.). Esta capacidad permite distribuir las responsabilidades por módulos sin obligar a mantener toda la aplicación en un único archivo.

La propuesta separa la interfaz web, los controladores Flask, los servicios de aplicación, los puertos y los adaptadores de infraestructura. Los controladores reciben solicitudes y delegan el trabajo a servicios como AuthService, InventarioService, ReparacionService e ImportacionService. Estos servicios se apoyan en contratos internos, por ejemplo, los repositorios y AIProvider, en lugar de depender directamente de SQLite o de una API externa. Este enfoque sigue el principio de puertos y adaptadores: la lógica de la aplicación se mantiene separada de los detalles técnicos de entrada y salida (Cockburn, 2005).

4.2 Componentes propuestos

Tabla 2 Componentes propuestos

Componente	Responsabilidad
Interfaz web	Presentar formularios, listados de productos, movimientos de inventario y pantallas de seguimiento.
Controladores Flask	Recibir solicitudes, verificar sus parámetros, invocar los servicios correspondientes y construir las respuestas.
Servicio de inventario	Aplicar las reglas de registro, actualización y movimiento de productos.

Servicio de reparaciones	Crear órdenes, administrar estados y registrar cambios en el historial.
Servicio de importación	Leer archivos de productos, normalizar campos y preparar los datos para validación.
Adaptador de IA	Enviar solicitudes al proveedor seleccionado y convertir sus respuestas al formato esperado por GestorTech.
Contrato AIProvider	Definir la operación común que debe cumplir cualquier proveedor de IA integrado.
Repositorios o capa de acceso a datos	Encapsular las operaciones de consulta y persistencia en la base de datos.

La tabla describe responsabilidades de diseño. La implementación concreta podrá variar según las decisiones técnicas que se adopten durante el desarrollo.

4.3 Separación de responsabilidades

La propuesta evita que los controladores web concentren todas las reglas del negocio. Por ejemplo, el controlador encargado de registrar una salida de inventario recibirá la solicitud y delegará la operación al servicio correspondiente. Este servicio verificará las reglas definidas antes de solicitar la persistencia de los cambios.

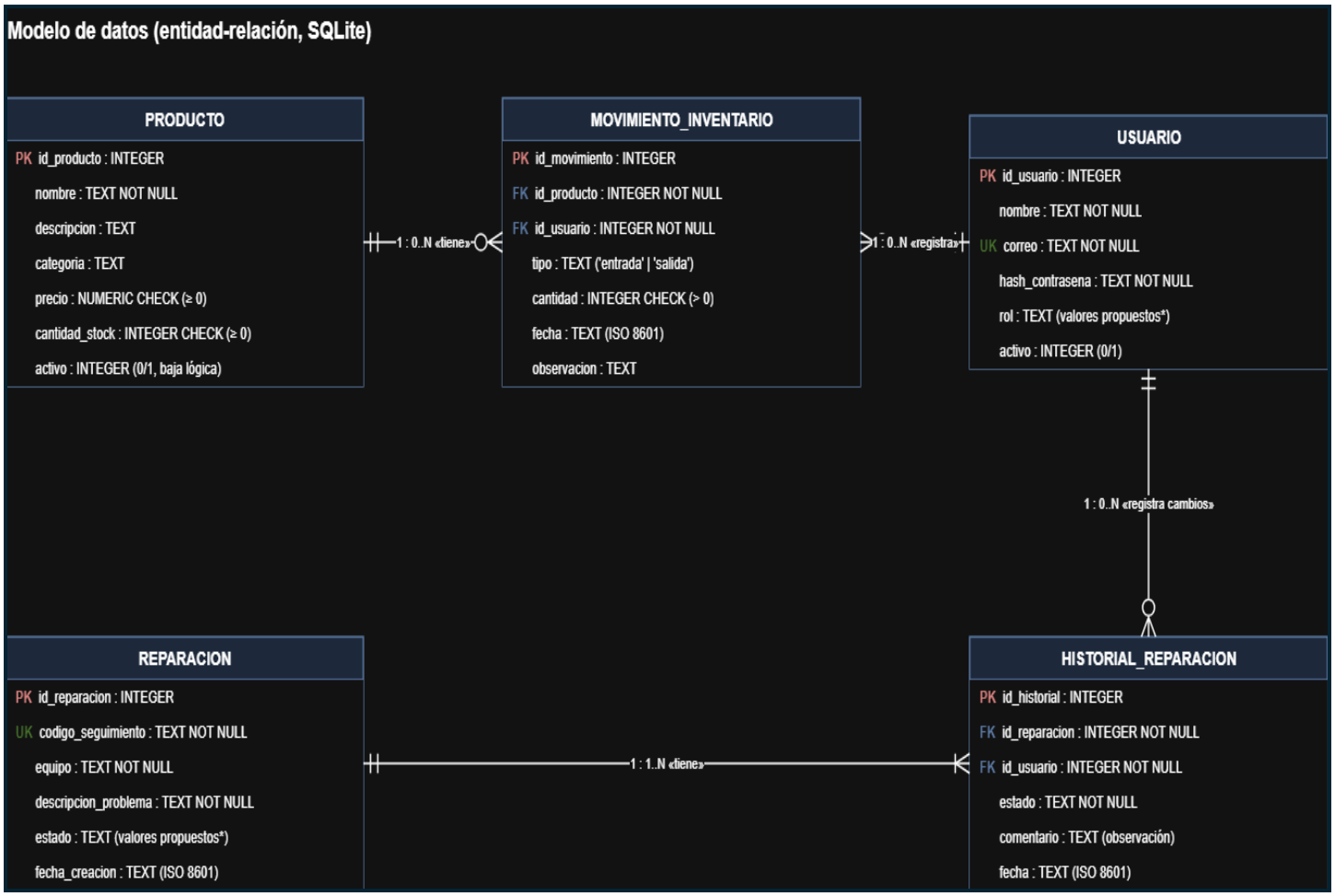
En el diagrama, Groq aparece como candidato inicial y OpenRouter como alternativa, ambos se conectan mediante adaptadores separados que implementan el contrato AIProvider. También se contempla NullAIProvider para mantener el modo sin IA y apoyar las pruebas. El archivo CSV/XLSX se procesa mediante ArchivoParser, mientras que los repositorios SQLite encapsulan la persistencia. La configuración de las claves de API se mantiene en variables de

entorno. Estas piezas son componentes propuestos y no deben interpretarse como código ya implementado.

La separación entre las reglas del negocio y los detalles de acceso a datos permite mantener más delimitadas las responsabilidades de cada componente. Fowler (2002) documenta patrones que abordan la organización de la lógica de aplicaciones y el acceso a los datos, su uso específico en GestorTech constituye una decisión de diseño del proyecto.

5. MODELO DE DATOS

Figura 3 Modelo de datos



5.1 Entidades principales

El diagrama de datos define cinco entidades: USUARIO, PRODUCTO, MOVIMIENTO_INVENTARIO, REPARACION e HISTORIAL_REPARACION. PRODUCTO conserva el nombre, descripción, categoría, precio, cantidad en stock y estado activo. MOVIMIENTO_INVENTARIO registra entradas o salidas, cantidad, fecha, observación y referencias al producto y al usuario que realiza el movimiento. USUARIO incluye correo único, hash de contraseña, rol y estado activo. REPARACION contiene un código de seguimiento único, el equipo, la descripción del problema, el estado y la fecha de creación. HISTORIAL_REPARACION conserva los cambios de estado, comentarios, fecha y usuario relacionado.

Tabla 3 Entidades principales

Entidad	Propósito	Atributos orientativos
USUARIO	Identificar a las personas autorizadas para utilizar el área administrativa.	id_usuario, nombre, correo, hash_contrasena, rol, activo
PRODUCTO	Registrar los productos y repuestos del establecimiento.	id_producto, nombre, descripcion, categoria, precio, cantidad_stock, activo
MOVIMIENTO_INVENTARIO	Mantener el registro de las entradas y salidas de productos.	id_movimiento, id_producto, id_usuario, tipo, cantidad, fecha, observacion
REPARACION	Representar una orden de servicio técnico.	id_reparacion, codigo_seguimiento, equipo, descripcion_problema, estado, fecha_creacion

HISTORIAL_REPARACION	Conservar los cambios de estado y sus observaciones.	id_historial, id_reparacion, id_usuario, estado, comentario, fecha

El diagrama propone relaciones de uno a muchos: un producto puede tener varios movimientos; un usuario puede registrar distintos movimientos; una reparación puede tener varios elementos de historial; y un usuario puede registrar cambios en reparaciones. El esquema no incluye por ahora una entidad independiente de cliente ni datos de contacto del cliente en REPARACION. Por ello, la consulta pública se plantea mediante el código de seguimiento y debe mostrar solo el historial y los campos expresamente autorizados. Si más adelante se requiere almacenar datos del cliente, esa necesidad deberá reflejarse en el modelo antes de implementarse.

5.2 Relaciones previstas

- Un producto puede tener varios movimientos de inventario.
- Un usuario autorizado puede registrar distintos movimientos.
- Una reparación puede contener varios registros en su historial.
- Un usuario autorizado puede registrar cambios en las reparaciones, según los permisos asignados.

Las relaciones se representarían mediante claves primarias y foráneas cuando corresponda. SQLite dispone de soporte para restricciones de clave foránea; su aplicación debe configurarse y

verificarse expresamente, ya que no basta con asumir que estas restricciones están activas en todas las conexiones (SQLite, s. f.-b).

5.3 Integridad de los datos

El diseño deberá contemplar las siguientes reglas:

- Los productos tendrán identificadores únicos.
- Las cantidades de movimientos deberán ser positivas.
- Las salidas de inventario deberán respetar la regla de disponibilidad definida para el negocio.
- El código de seguimiento de cada reparación deberá ser único.
- Los cambios de estado de una reparación deberán registrarse en el historial.
- Los campos que representen dinero deberán manejarse con una estrategia de almacenamiento y cálculo que evite errores de precisión.
- Las operaciones relacionadas entre sí deberán persistirse de forma coherente.

SQLite documenta sus características y particularidades de almacenamiento y consultas en su documentación oficial (SQLite, s. f.-a). Por tanto, la definición final del esquema debe considerar las características concretas del motor y no depender únicamente de las convenciones utilizadas en otros sistemas de bases de datos. Se plantea como una opción para la primera versión por su sencillez de integración. La elección es una decisión de diseño, no una afirmación de que sea la alternativa apropiada para cualquier volumen de usuarios o patrón de concurrencia.

6. ESPECIFICACIÓN INICIAL DE LA API

6.1 Propósito

La API permitirá que la interfaz web intercambie información con los servicios del sistema mediante solicitudes HTTP y respuestas estructuradas. Los endpoints presentados a continuación constituyen una especificación preliminar; los métodos, permisos, códigos de respuesta y esquemas deberán confirmarse durante la implementación.

La especificación OpenAPI permite describir interfaces HTTP mediante una definición independiente del lenguaje de programación, incluyendo operaciones, parámetros, cuerpos de solicitud y respuestas (OpenAPI Initiative, 2025).

6.2 Endpoints propuestos

Tabla 4 Endpoints

Método	Endpoint	Función	Acceso previsto
GET	/api/productos	Consultar productos.	Personal autorizado; una consulta pública, si se habilita, deberá tener una ruta y campos limitados.
POST	/api/productos	Registrar un producto.	Personal autorizado.
PUT	/api/productos/{id}	Actualizar los datos de un producto existente.	Personal autorizado.

GET	/api/productos/{id}/movimientos	Consultar movimientos asociados a un producto.	Personal autorizado.
POST	/api/productos/{id}/movimientos	Registrar una entrada o salida de inventario.	Personal autorizado.
POST	/api/reparaciones	Crear una orden de reparación.	Personal autorizado.
PATCH	/api/reparaciones/{id}/estado	Actualizar el estado de una reparación y registrar el cambio.	Personal autorizado.
GET	/api/seguimiento/{codigo}	Consultar el estado público de una reparación mediante su código.	Consulta pública limitada.
POST	/api/ia/importacion/propuesta	Solicitar una propuesta de organización de datos de productos.	Personal autorizado.

La consulta pública de seguimiento no deberá devolver contraseñas, datos internos del negocio, información de otros clientes ni comentarios que no estén destinados a consulta pública. La existencia de un código de seguimiento no sustituye el análisis de qué información puede exponerse.

6.3 Ejemplo de solicitud para registrar un producto

El siguiente ejemplo ilustra el formato propuesto:

{

"nombre": "Cable USB-C",
"descripcion": "Cable de carga USB-C",
"categoria": "Accesorios",
"precio": 18000,
"cantidad_stock": 12
}

La aplicación deberá validar los campos recibidos antes de guardar la información. La estructura del ejemplo no determina por sí sola los tipos de datos, las reglas monetarias ni las restricciones definitivas de la implementación.

6.4 Ejemplo de creación de una reparación

{
"equipo": "Computador portátil",
"descripcion_problema": "El equipo no inicia correctamente"
}

La respuesta de creación podría contener el identificador interno, el código de seguimiento y el estado inicial asignado por el sistema. La generación del código y el estado inicial deberán definirse en la lógica del negocio.

6.5 Ejemplo de consulta de seguimiento

```
{
 "codigo_seguimiento": "GT-000123",
 "estado": "En diagnóstico",
 "ultima_actualizacion": "2026-10-20T14:30:00",
 "historial_publico": [
  {
   "estado": "Recibido",
   "comentario": "Equipo recibido para revisión"
  },
  {
   "estado": "En diagnóstico",
   "comentario": "Se está revisando el equipo"
  }
 ]
}
```


Los valores son ilustrativos y no corresponden a una reparación real. La respuesta definitiva deberá incluir únicamente los campos autorizados para el cliente y establecer de forma consistente el formato de fecha y zona horaria.

6.6 Respuestas y errores

Se propone que la API utilice códigos HTTP coherentes con el resultado de cada operación. Por ejemplo, una creación exitosa podría responder con 201 Created; una solicitud con datos inválidos, con 400 Bad Request o 422 Unprocessable Content, según la convención adoptada; y una consulta sin un recurso correspondiente, con 404 Not Found.

La selección definitiva de códigos deberá documentarse junto con cada endpoint. También se definirán mensajes de error que permitan comprender el problema sin revelar detalles internos del servidor.

7. DECISIONES ARQUITECTÓNICAS

Tabla 5 Decisiones arquitectónicas

Decisión	Justificación	Consideración o limitación
Utilizar Python y Flask	Permite desarrollar la aplicación web con una estructura modular.	Requiere definir la organización del código, la configuración y los controles de seguridad.
Utilizar HTML, CSS y JavaScript	Permite construir las pantallas del área administrativa y de consulta.	La interfaz deberá validar la experiencia de uso y manejar correctamente los errores del servidor.
Utilizar SQLite en la primera versión	Reduce la complejidad inicial de instalación y persistencia.	Se debe evaluar su adecuación según la concurrencia, el volumen y el despliegue.
Separar controladores, servicios y acceso a datos	Delimita responsabilidades y facilita localizar las reglas del negocio.	La separación debe corresponder a necesidades reales y no convertirse en una complejidad innecesaria.
Exponer operaciones mediante una API HTTP	Define una forma estructurada de comunicación entre la interfaz y el backend.	Deben documentarse permisos, validaciones, respuestas y errores.

Integrar la IA mediante un adaptador	Evita que el código del negocio dependa directamente de un proveedor particular.	Cada proveedor puede presentar diferencias en formato, capacidades, disponibilidad y costos.
Exigir revisión humana antes de guardar propuestas de IA	Mantiene la decisión final en manos de una persona autorizada.	La revisión debe comprobar los campos críticos y no limitarse a aceptar la propuesta completa.
Permitir que las funciones esenciales operen sin IA	Evita que la disponibilidad del servicio externo sea un requisito para administrar inventario y reparaciones.	La aplicación deberá definir cómo informa al usuario cuando la asistencia de IA no está disponible.

La organización modular de Flask puede apoyarse en sus mecanismos de Blueprints, descritos en la documentación oficial (Pallets Projects, s. f.). La separación entre lógica y persistencia se relaciona con patrones de organización de aplicaciones descritos por Fowler (2002), mientras que la separación mediante puertos y adaptadores se fundamenta en el enfoque propuesto por Cockburn (2005). La aplicación concreta de estas ideas corresponde a las decisiones de diseño de GestorTech.

8. DISEÑO DE LA INTEGRACIÓN DE IA

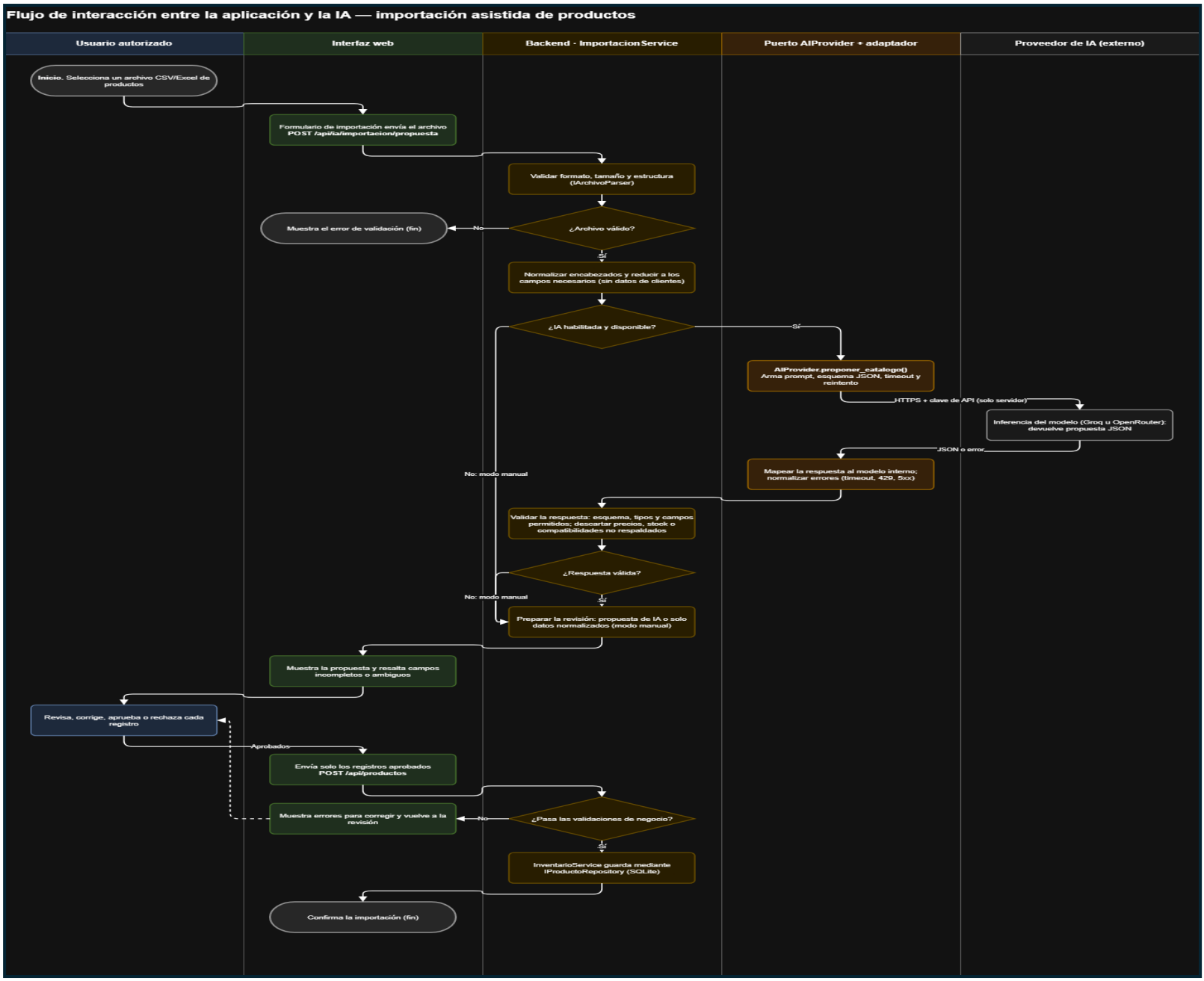
8.1 Objetivo de la integración

La IA se propone como apoyo para preparar la información de productos antes de incorporarla al catálogo. La función no reemplazará las reglas de validación del sistema ni será responsable de decidir los valores definitivos del inventario.

La utilidad esperada consiste en reducir parte del trabajo de organización de archivos que contienen nombres, descripciones o categorías de productos en columnas diferentes o con formatos inconsistentes.

8.2 Flujo propuesto

Figura 4 Flujo de importación asistida de productos mediante IA



El diagrama detalla el proceso de importación desde que una persona autorizada selecciona un archivo CSV o Excel hasta que se guardan únicamente los registros aprobados. Primero, el backend valida formato, tamaño y estructura mediante IArchivoParser. Después normaliza encabezados y limita los campos enviados, excluyendo datos de clientes. Si la IA está deshabilitada o no disponible, el flujo continúa en modo manual. Si está disponible, AIProvider solicita una propuesta estructurada; el backend normaliza errores, valida el esquema y descarta valores de precio, stock o compatibilidad que no tengan respaldo. La interfaz presenta la propuesta para revisión; solo los registros aprobados pasan por las validaciones del InventarioService y se guardan mediante IProductoRepository en SQLite.

- Una persona autorizada carga un archivo compatible con el formato definido para la importación.
- El sistema comprueba el tipo de archivo, su estructura y los límites establecidos.
- El servicio de importación normaliza los encabezados y prepara los campos necesarios.
- Si la asistencia de IA está habilitada, el adaptador envía los datos mínimos necesarios al proveedor seleccionado.
- El proveedor devuelve una propuesta estructurada.
- El backend valida la estructura y los valores recibidos.
- La interfaz presenta los resultados para que una persona autorizada los revise.
- Los datos aprobados pasan por las validaciones de negocio y solo entonces se guardan en la base de datos.

8.3 Datos que puede proponer

La asistencia de IA podría utilizarse para:

- Identificar posibles equivalencias entre encabezados de columnas.
- Proponer una organización de los campos de un archivo.
- Normalizar descripciones a partir de los datos de origen.
- Sugerir una categoría cuando la información disponible permita formular una propuesta.
- Señalar campos incompletos o ambiguos para revisión.

La precisión de estas propuestas no se da por garantizada. Los resultados deberán comprobarse con archivos representativos antes de decidir si la función ofrece suficiente utilidad para incorporarse a la versión operativa.

8.4 Datos que no debe inventar

Los precios, las cantidades de inventario y las compatibilidades de productos son datos que requieren respaldo en la fuente original o confirmación de una persona autorizada.

Si un archivo no contiene un precio, el sistema no deberá permitir que la IA complete ese campo con un valor supuesto. Si no se conoce la compatibilidad de un repuesto, no se presentará una sugerencia como si fuera un hecho verificado.

La aplicación podrá marcar estos campos como pendientes de revisión, conservarlos vacíos cuando el modelo de datos lo permita o solicitar que el usuario los complete. La decisión definitiva dependerá de las reglas de negocio establecidas para cada campo.

8.5 Contrato de respuesta

La integración deberá solicitar resultados estructurados y comprobar su conformidad antes de mostrarlos como propuestas válidas. Un ejemplo sería:


```
{
  "propuesta": [
    {
      "nombre": "Cable USB-C",
      "descripcion": "Cable de carga USB-C",
      "categoria": "Accesorios",
      "precio": null,
      "cantidad_stock": null,
      "campos_pendientes": [
        "precio",
        "cantidad_stock"
      ]
    }
  ],
  "requiere_revision_humana": true
```

}

En este ejemplo se define el formato posible para distinguir los campos propuestos de aquellos que necesitan confirmación. El backend deberá comprobar los tipos de datos, los campos permitidos y las reglas del negocio, independientemente de lo que devuelva el servicio externo.

8.6 Gestión de riesgos y supervisión

La incorporación de IA introduce riesgos relacionados con respuestas incorrectas, información incompleta, exposición innecesaria de datos y dependencia de un servicio externo.

El Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile del National Institute of Standards and Technology examina riesgos asociados con la IA generativa y ofrece orientación para incorporarlos a actividades de diseño, desarrollo, uso y evaluación (Autio et al., 2024). Para este proyecto, esta orientación sirve como referencia para definir controles, no constituye una certificación ni demuestra por sí misma que la integración sea segura.

Se proponen las siguientes medidas:

- Enviar al proveedor únicamente los campos necesarios para la tarea.
- Evitar incluir datos personales de clientes en solicitudes de organización del catálogo.
- Validar la respuesta antes de presentarla como una propuesta utilizable.
- Mantener la revisión humana obligatoria antes de guardar los registros.
- Registrar los errores de integración sin almacenar innecesariamente contenido sensible.

- Establecer límites para el tamaño de los archivos y la cantidad de registros procesados.
- Informar al usuario cuando el servicio no responda o produzca resultados que no cumplan el formato esperado.

La efectividad de estas medidas tendrá que evaluarse durante el desarrollo y las pruebas.

9. SUSTITUCIÓN DEL PROVEEDOR DE IA Y CONTINUIDAD OPERATIVA

9.1 Interfaz independiente del proveedor

Se propone definir un contrato denominado AIProvider que represente la operación de asistencia necesaria para preparar datos de productos. El contrato deberá especificar qué información recibe, qué formato de respuesta entrega y cómo comunica los errores.

El adaptador de cada proveedor será responsable de convertir las solicitudes y respuestas entre el formato interno de GestorTech y el formato requerido por el servicio externo.

De esta forma, los controladores y servicios del negocio dependerán de una interfaz interna, no de los detalles particulares de un proveedor. Esta es una decisión de diseño inspirada en la separación de responsabilidades y en la inversión de dependencias descrita en la literatura de diseño de software de Martin (2003).

9.2 Comparación de proveedores y análisis costo-beneficio preliminar

La función de inteligencia artificial de GestorTech tiene un propósito acotado: ayudar a organizar los campos de archivos CSV o Excel antes de importar productos al inventario. Por ello, el proveedor debe evaluarse según su capacidad para devolver información estructurada, la calidad de las propuestas, el costo por uso, la facilidad de integración y las condiciones de disponibilidad y privacidad. No se requiere que el proveedor gestione inventario ni que tenga acceso directo a la base de datos.

Se consideran Groq y OpenRouter como alternativas para una evaluación inicial. Groq ofrece acceso mediante API a modelos alojados en su plataforma; OpenRouter ofrece una API unificada para acceder a modelos de distintos proveedores. La comparación siguiente es preliminar y no sustituye pruebas con archivos representativos del negocio.

Tabla 6 Proveedores costo-beneficio

Criterio	Groq	OpenRouter
Enfoque del servicio	API de inferencia para modelos disponibles en GroqCloud.	API unificado para utilizar modelos de distintos proveedores.
Relación con la tarea de GestorTech	Puede recibir los campos necesarios y generar una propuesta estructurada para organizar productos.	Puede realizar la misma tarea seleccionando un modelo compatible entre las opciones disponibles.
Costos	Se calculan según el modelo y el consumo de tokens de entrada y salida. La tarifa publicada debe verificarse al seleccionar el modelo.	Se calculan según el modelo y el consumo. En el plan estándar de pago por uso, OpenRouter publica una tarifa de plataforma del 5,5 % al comprar créditos, además del costo de inferencia del modelo.

Beneficio potencial	Permite probar modelos dentro de una misma plataforma y evaluar su velocidad, calidad y costo para el volumen previsto.	Facilita comparar o cambiar modelos desde una interfaz común, lo que aporta flexibilidad durante la evaluación.
Limitaciones y riesgos	La aplicación dependerá de los modelos disponibles, sus límites, precios y condiciones de servicio.	La calidad, disponibilidad y precio dependen también del modelo y del proveedor que procese cada solicitud; la tarifa de plataforma debe incluirse en la estimación.
Costo-beneficio para el MVP	Puede resultar conveniente si un modelo disponible cumple la estructura y calidad requeridas a un costo adecuado.	Puede resultar conveniente si la flexibilidad para comparar modelos aporta valor suficiente frente a los costos y la complejidad de selección.
Estado de la evaluación	Candidato propuesto para pruebas iniciales; no validado todavía.	Alternativa para comparación; no validada todavía.

Las tarifas y los modelos disponibles pueden cambiar. La documentación oficial de Groq publica precios específicos por modelo y por millón de tokens, mientras que OpenRouter indica que el costo de inferencia depende del modelo y que su plan estándar aplica una tarifa del 5,5 % al comprar créditos (Groq, s. f.; OpenRouter, s. f.). Por este motivo, no se fija aquí un costo mensual definitivo: todavía no se ha establecido el volumen real de importaciones ni el consumo promedio de tokens por producto.

Se propone evaluar primero Groq y mantener OpenRouter como alternativa de comparación. Esta prioridad se plantea como una decisión de evaluación inicial, no como una conclusión de que Groq sea definitivamente más económico o más preciso. El documento no

registra todavía pruebas comparativas que permitan afirmar cuál ofrece el mejor resultado para GestorTech.

La decisión definitiva se tomará después de probar ambos servicios, cuando sea posible, con un conjunto representativo de filas de inventario. Se compararán: (1) proporción de respuestas con formato válido; (2) exactitud al conservar los datos originales; (3) capacidad para señalar campos incompletos sin inventarlos; (4) tiempo de respuesta; y (5) costo estimado para un lote y un volumen mensual definidos. Se descartará cualquier configuración que complete con valores supuestos precios, cantidades o compatibilidades.

El costo-beneficio no se medirá únicamente por la tarifa de tokens. También se tendrá en cuenta el tiempo de revisión que ahorre o añada la propuesta de IA, los errores que obligue a corregir y el trabajo necesario para mantener la integración. Si la IA no ofrece una mejora suficiente, la importación manual seguirá disponible.

9.3 Elementos que podrían cambiar durante una sustitución

Tabla 7 Elementos que podrían cambiar

Elemento	¿Podría cambiar?	Motivo
Adaptador del proveedor	Sí	Groq y OpenRouter pueden diferir en sus endpoints, autenticación y parámetros de solicitud.
Configuración y credenciales	Sí	Cada servicio requiere credenciales y variables de configuración propias.

Identificador y parámetros del modelo	Sí	Los modelos tienen identificadores, capacidades y límites diferentes.
Tratamiento de errores	Sí	Los proveedores pueden devolver códigos y estructuras de error distintos.
Contrato interno AIProvider	Preferiblemente no, salvo necesidad justificada	Busca mantener estable la forma en que el resto del sistema solicita asistencia.
Servicios de importación, inventario y reparaciones	No deberían requerir cambios por una sustitución ordinaria	No deben depender directamente de la API de un proveedor externo.
Modelo de datos del inventario	No debería cambiar por una sustitución ordinaria	Los datos del negocio deben mantener independencia frente al servicio externo.

La tabla expresa el objetivo de la arquitectura, no una garantía de que nunca serán necesarios ajustes. Si un proveedor no admite alguna capacidad requerida, será necesario revisar el adaptador o las funcionalidades dependientes. La sustitución también deberá someterse a pruebas antes de utilizarse.

9.4 Continuidad sin IA

La IA será opcional. Si el servicio no está disponible, el usuario deberá poder seguir registrando productos de manera manual, consultando el inventario y administrando las reparaciones.

La aplicación podrá mostrar un aviso de indisponibilidad y permitir que el usuario continúe por el flujo manual. Los reintentos, si se implementan, deberán ser limitados para evitar solicitudes repetidas innecesarias.

Este comportamiento reduce el acoplamiento operativo entre la gestión principal del establecimiento y un servicio externo cuya disponibilidad no controla GestorTech.

10. SEGURIDAD, PRIVACIDAD Y CONTROL DE ACCESO

La seguridad debe considerarse desde el diseño de la aplicación y no únicamente cuando se despliegue. La API puede exponer información y operaciones del negocio, por lo que cada endpoint deberá tener reglas de acceso y validaciones acordes con su finalidad.

OWASP identifica riesgos de seguridad para las API, entre ellos problemas de autorización a nivel de objeto, autenticación, permisos sobre propiedades y consumo no restringido de recursos (OWASP Foundation, 2023). Estos riesgos sirven como referencia para revisar los endpoints propuestos.

Se establecen las siguientes consideraciones de diseño:

- **Autenticación:** las operaciones administrativas requerirán identificar al usuario.
- **Autorización:** cada operación deberá comprobar que el usuario cuenta con los permisos necesarios.
- **Contraseñas:** se almacenarán mediante un mecanismo de hash apropiado; no se guardarán en texto plano.
- **Validación de entradas:** el backend verificará los tipos, los formatos y las reglas de los datos recibidos.
- **Protección de consultas públicas:** el seguimiento de reparaciones mostrará solo los campos autorizados para el cliente.
- **Protección de credenciales:** las claves del proveedor de IA no deberán incluirse en el código fuente ni exponerse en la interfaz.
- **Control de archivos:** la importación tendrá límites de tamaño y validaciones del contenido.
- **Manejo de errores:** las respuestas no revelarán credenciales, consultas internas ni detalles sensibles del servidor.
- **Protección de datos:** se enviará al servicio externo de IA únicamente la información necesaria para preparar los productos.

Estas medidas constituyen requisitos de diseño que deberán traducirse en controles concretos y verificarse mediante pruebas.

11. VALIDACIÓN PREVISTA DEL DISEÑO

La arquitectura propuesta deberá revisarse a medida que avance la implementación. Para ello se plantean las siguientes actividades:

Tabla 8 Validación prevista del diseño

Actividad	Propósito	Evidencia esperada
Revisión del diagrama de contexto	Comprobar que los actores y sistemas externos coincidan con el alcance definido.	Diagrama actualizado y coherente con el documento.
Revisión del diagrama de componentes	Verificar que las responsabilidades representadas correspondan a los módulos propuestos.	Diagrama revisado.
Revisión del modelo de datos	Comprobar las relaciones, las restricciones y los campos necesarios para el alcance inicial.	Modelo de datos ajustado.
Revisión de la API	Comprobar que cada endpoint tenga definidos propósito, permisos, entradas y respuestas.	Especificación inicial documentada.
Pruebas funcionales	Verificar el registro de productos, los movimientos de inventario y el seguimiento de reparaciones cuando se implementen.	Casos de prueba y resultados obtenidos.
Pruebas de la integración de IA	Comprobar respuestas válidas, manejo de errores y necesidad de revisión humana.	Registro de pruebas y errores encontrados.
Revisión de seguridad	Examinar permisos, validación de datos y exposición de información.	Lista de hallazgos y medidas correctivas.

Las actividades que todavía no se hayan ejecutado deberán mantenerse como pendientes.

12. CONCLUSIONES

La arquitectura propuesta para GestorTech organiza las responsabilidades necesarias para gestionar productos, registrar movimientos de inventario y consultar el estado de reparaciones técnicas. La separación entre la interfaz, los servicios del negocio, el acceso a datos y la integración de IA establece una base para desarrollar cada función de manera delimitada.

El modelo de datos y la API inicial permiten describir cómo se relacionarán las entidades y cómo se comunicarán los componentes de la aplicación. Sin embargo, ambos deberán revisarse durante la implementación para confirmar que responden a las reglas reales del negocio y que las restricciones de integridad y acceso estén correctamente definidas.

La integración de inteligencia artificial se limita a una función de asistencia para preparar información de productos. La propuesta mantiene la revisión humana antes del guardado, evita que la IA tenga acceso directo a la base de datos y establece que los procesos principales puedan continuar cuando el proveedor externo no esté disponible.

Por último, la documentación de referencia aporta fundamentos para las decisiones de arquitectura, persistencia, diseño de API y gestión de riesgos. La calidad final de la solución dependerá de que estas decisiones se traduzcan en código, controles y pruebas que permitan comprobar su funcionamiento.

REFERENCIAS BIBILIOGRÁFICAS

Autio, C., Schwartz, R., Dunietz, J., Jain, S., Stanley, M., Tabassi, E., & Hall, P. (2024). Artificial intelligence risk management framework: Generative artificial intelligence profile (NIST AI 600-1). National Institute of Standards and Technology. *https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf*

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley Professional. *https://martinfowler.com/books/eaa.html*

Cockburn, A. (2005). Hexagonal architecture. *https://alistair.cockburn.us/hexagonal-architecture*

Brown, S. (s. f.). C4 model: Diagrams. *https://c4model.com/diagrams*

Martin, R. C. (2003). Agile software development: Principles, patterns, and practices. Prentice Hall. *https://ebooks.karbust.me/Technology/Agile.Software.Development.Principles.Patterns.and.Practices.Pearson.pdf*

OpenRouter. (s. f.). *Pricing*. *https://openrouter.ai/pricing*

Groq. (s. f.). *Supported models and pricing*. *https://console.groq.com/docs/models*

OpenAPI Initiative. (2026). OpenAPI specification, version 3.2.1. *https://spec.openapis.org/oas/v3.2.1.html*

OWASP Foundation. (2023). OWASP API security top 10—2023. *https://api-security.owasp.org/*

Pallets Projects. (s. f.). Flask documentation. *https://flask.palletsprojects.com/en/stable/*

SQLite. (s. f.-a). SQLite documentation. https://www.sqlite.org/docs.html

SQLite. (s. f.-b). SQLite foreign key support. Recuperado el 27 de octubre de 2026, de https://www.sqlite.org/foreignkeys.html